

AI-ASSISTED TRADING AUTOMATION

From Trading Idea to MT5 Indicator

A Practical, Step-by-Step Guide to Turning a Trading Idea Into Working MT5 Code Using AI

Introduction

You do not need to be a programmer to start automating your trading strategy.

The first step is not writing code. The first step is explaining what you want to build.

AI can help you take a simple trading idea and turn it into a clear specification that a coding AI can understand. The process is simple:

IDEA !' SPECIFICATION !' CODING PROMPT !' CODE !' MT5 !' TEST !' IMPROVE

In this guide we use one worked example — an MT5 Average Daily Range (ADR) indicator — to demonstrate the full process from idea to finished tool.

The goal here is not to teach you how ADR works as a trading concept. The goal is to show you how to take an idea from your head and use AI to turn it into a working MT5 indicator — the same indicator shown running on a live chart in Figure 1 below.

Part 1 — Start With the Idea

When you have an idea for an indicator or Expert Advisor, don't start by thinking about programming. Start by asking:

“What do I want this tool to do?”

You can explain your idea in normal language. You don't need to know MQL5. You don't need to know programming terminology. You simply describe what you want to see and how you want the tool to behave.

Case Study: The ADR Indicator

Imagine you want an MT5 indicator that calculates ADR using a number of previous days and displays the expected levels on your chart. Instead of trying to write the code yourself, you start by giving ChatGPT a simple, rough request.

STEP 1 — YOUR ROUGH IDEA (GIVEN TO CHATGPT)

Create specifications for me to tell AI to build an MT5 ADR indicator that is based on X number of days. It's calculated from the opening price of the day to the high or low of the day, and it should draw dotted lines every day after market open, and when it's reached, it should give a notification, and it should also show the number of days used. I want a notification on screen when reached, and it should show the number of pips to reach the level. Make it clear and simple to understand.

That is enough to begin. You don't need to make the request perfect — you are simply giving AI your initial idea.

Part 2 — Let ChatGPT Help Structure the Idea

Now ChatGPT's job is to take your rough idea and turn it into a clearer specification. It can help identify things such as:

- What information the indicator needs from you.
- What it should calculate.
- What should appear on the chart.
- What should happen when a level is reached.
- What notifications are required.
- What information should be displayed to you.

This is where AI becomes your specification assistant. You are not asking it to decide your trading strategy — you are asking it to help you clearly describe your strategy or idea.

Part 3 — Review What AI Gives You

This is one of the most important parts of the process. Do not automatically accept everything AI gives you.

Read through the specification carefully and ask yourself: "Is this actually what I want?"

AI may misunderstand something. It may add something you don't want. It may leave something out. It may interpret your idea differently from how you intended. That is normal — your job is to review the specification.

Part 4 — Accept, Modify or Delete

When reviewing the AI response, you have three simple choices:

Accept

If AI understood your idea correctly, keep it.

Modify

If something is almost correct but needs to change, tell AI what you want changed.

EXAMPLE — MODIFY

Change this part so that the indicator uses 10 previous completed days instead of 5.

Delete

If AI added something you don't want, remove it.

EXAMPLE — DELETE

Remove the historical chart display. I only want the current day's levels.

You can continue doing this until the specification accurately represents your idea.

Part 5 — Finalize the Specification

Once you have reviewed everything, you should have a final specification. This becomes the blueprint for your indicator.

At this point, you should be able to read the specification and clearly understand:

- What the indicator does.
- What information it uses.
- What appears on the chart.
- What happens when a level is reached.
- What notifications are required.
- What settings the user can control.

This is an important milestone. You have gone from “I have an idea” to “I know exactly what I want the software to do.”

Part 6 — Don't Ask Claude to Code Yet

Now we move to the next stage. Instead of taking your original rough request directly to a coding AI, use ChatGPT to prepare a proper coding prompt.

EXAMPLE PROMPT TO CHATGPT

Now take this finalized specification and create a detailed coding prompt for Claude to build this MT5 indicator in MQL5.

ChatGPT can then turn your finalized specification into instructions specifically designed for the coding AI.

Part 7 — Review the Coding Prompt

Before sending the prompt to Claude, review it. Again, you are the person in control.

- Make sure the prompt accurately describes what you want.
- If something is wrong, go back and change it.
- If something is missing, add it.
- If something unnecessary was added, remove it.

Only when you are satisfied should you send the prompt to Claude.

Part 8 — Give the Prompt to Claude

Now Claude becomes your coding assistant. Give Claude the finalized coding prompt, and Claude creates the MQL5 code required for the MT5 indicator. This is where the idea finally becomes actual software.

Notice the order: you did not start with code. You started with the trading idea. Then you created the specification. Then you created the coding instructions. Only then did you generate the code.

The complete MQL5 source Claude produced for this case study is included in full in Appendix A, and referenced again once the finished indicator is on the chart in Part 9.

Part 9 — Move the Code Into MT5

Once Claude gives you the MQL5 code, move it into MetaEditor. Create the appropriate MT5 indicator file, place the code inside it, then compile it.

At this stage, something important can happen: the code may compile successfully, or MetaEditor may show errors. Both situations are normal.

Figure 1 shows the finished indicator from this case study running live on an MT5 chart. Notice how every element traces directly back to the original rough idea from Part 1: the dotted ADR High/Low lines projected from the daily open, the “REACHED” status once a level is hit, and the live pips-remaining readout in the on-chart panel.

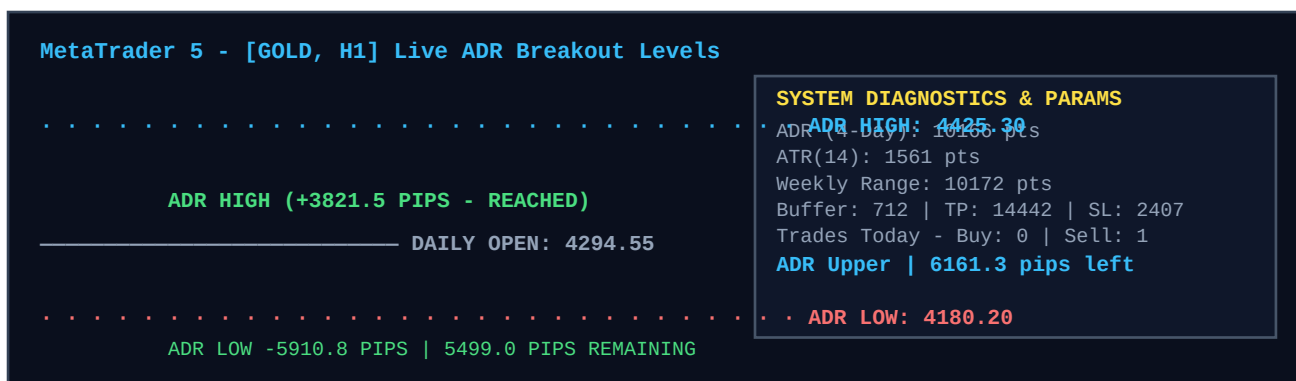


Figure 1 — The finished ADR indicator running on an MT5 chart, showing the daily-open reference line, the dotted ADR High/Low breakout levels, live REACHED status, and pips remaining to each level.

The complete MQL5 code behind this exact behaviour is in Appendix A — “Complete MQL5 Source Code: ADR_Levels.mq5.”

Part 10 — If There Are Errors, Don't Guess

If MetaEditor shows errors, don't try to randomly change the code yourself. Instead, copy the exact errors shown by MetaEditor and take them back to Claude.

EXAMPLE — REPORTING AN ERROR

*MetaEditor produced the following errors. Please fix them and provide the complete corrected MQL5 code.
[paste the exact compiler error text here]*

Claude can use those errors to identify what needs to be corrected. After receiving the corrected code:

REPLACE THE OLD CODE !' COMPILE AGAIN !' CHECK THE RESULT

Repeat this process until the code compiles properly.

Part 11 — Compiling Is Not the End

A very important lesson: a successful compilation does not automatically mean your strategy is correct.

The code can compile without errors and still behave differently from what you intended. That is why you must test the indicator on an MT5 chart and ask:

- Does it display what I requested?
- Are the levels correct?
- Are the notifications working?
- Are the displayed values correct?
- Does it behave correctly when a level is reached?
- Does it reset correctly when required?
- Does it match my original specification?

You are testing the behaviour, not just the code.

Part 12 — Compare the Result With Your Original Idea

Go back to your finalized specification and compare it with the actual indicator. This is where you become the tester.

If something is wrong, explain the problem to Claude.

EXAMPLE — REPORTING A BEHAVIOUR ISSUE

The indicator is working, but the notification is appearing multiple times. I only want one notification when the level is reached.

Claude can then modify the code. You compile again. You test again. This process continues until the software behaves the way you originally intended.

The Complete AI Automation Workflow

The complete process, from a bare idea to a validated tool, looks like this:

1. Idea

You have a trading idea.

!“

2. ChatGPT

Explain the idea in normal language.

!“

3. Brainstorm

ChatGPT helps turn the idea into a structured specification.

!“

4. Review

Read what AI created.

!“

5. Accept / Modify / Delete

Keep what is correct. Change what is wrong. Remove what you don't want.

!“

6. Finalize

Create the final specification.

!“

7. Coding Prompt

Ask ChatGPT to turn the specification into a prompt for Claude.

!“

8. Review Prompt

Make sure Claude will understand exactly what needs to be built.

!“

9. Claude

Give the prompt to Claude.

!“

10. MQL5 Code

Claude generates the code (see Appendix A).

!“

11. MT5 / MetaEditor

Put the code into MetaEditor.

!“

12. Compile

Check whether the code compiles.

!“

13. Debug

If there are errors, send the exact errors back to Claude.

!“

!“

14. Test

Run the indicator on MT5 (see Figure 1).

!“

15. Validate

Compare the actual behaviour with your original specification.

!“

16. Improve

Make changes where necessary.

Who Does What?

Understanding the roles makes this process much easier.

You — The Strategist	• What the strategy does. • What the indicator or EA should do. • What rules should be followed. • What you want to see. • What should happen in different situations. • You are in control of the strategy.
ChatGPT — The Specification Assistant	• Explains your idea. • Organizes your thoughts. • Identifies missing details. • Creates a clear specification. • Turns the specification into a coding prompt.
Claude — The Coding Assistant	• Converts the specification into MQL5. • Corrects coding errors. • Modifies the code. • Improves the implementation based on your instructions.
MT5 — The Testing Environment	• Compiles the code. • Puts the tool on a chart. • Observes its behaviour. • Tests whether it works according to your specification.

The Most Important Principle

Do not think:

“I need to learn how to code before I can automate my strategy.”

Instead, think:

“I need to learn how to clearly explain what I want to build.”

AI can help with much of the technical work. But AI cannot replace your understanding of your own trading strategy. If you cannot clearly explain what your strategy should do, AI cannot magically know what you mean.

Three Golden Rules

Rule 1 — Idea First, Code Last

Never start by asking AI to write code when you haven't clearly defined what you want. Start with the idea. Turn the idea into a specification. Then create the code.

Rule 2 — Don't Blindly Trust AI

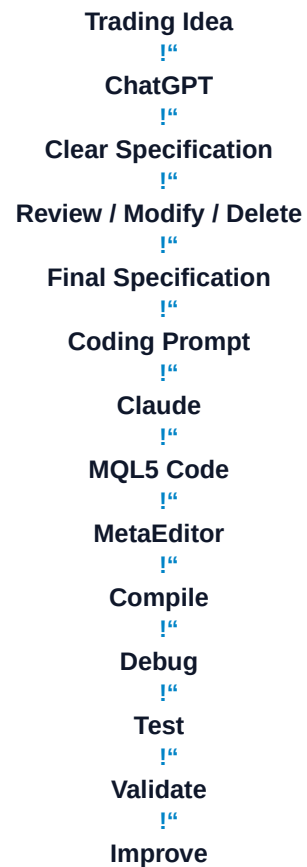
AI can misunderstand you. AI can make assumptions. AI can add things you never requested. Always review the output. You are the decision-maker.

Rule 3 — Test the Behaviour

A green compilation message is not proof that your strategy has been automated correctly. Always test the actual result against your original idea.

Final Framework

You can use this same process for almost any trading automation project:



The ADR indicator in this case study is simply an example. The real skill you are learning is how to take a trading idea from your mind and systematically turn it into working software with the help of AI.

Idea First. Specification Second. Code Last.

Appendix A — Complete MQL5 Source Code: ADR_Levels.mq5

This is the finished MQL5 code Claude generated for the ADR indicator case study used throughout this guide. It implements every point from the finalized specification in Part 5:

- An adjustable number of previous days (InpADRDays) used to calculate the ADR.
- ADR distances measured from each day's opening price to that day's high and low.
- Dotted High/Low lines projected from today's opening price, redrawn automatically at the start of each new trading day.
- An on-screen status panel showing the daily open, both ADR levels, and the live pips remaining to each level.
- A popup Alert(), an optional mobile push notification, and an optional sound the first time a level is reached.
- A REACHED status shown directly on the chart once a level is hit, matching Figure 1.

To use it: open MetaEditor, create a new Custom Indicator called ADR_Levels, paste the code below over the template, then compile (F7) and drag the indicator onto a chart.

```
//+-----+
//|                                     ADR_Levels.mq5 |
//|               Case Study: AI-Assisted Trading Automation |
//|   Average Daily Range (ADR) breakout-level indicator for MT5   |
//+-----+
#property copyright "Case Study - AI-Assisted Trading Automation"
#property link      ""
#property version   "1.00"
#property indicator_chart_window
#property indicator_plots 0

//--- Input parameters -----
input group "ADR Settings"
input int   InpADRDays      = 4;    // Number of previous completed days used for ADR
input double InpBufferPoints = 0;    // Extra buffer added to each level (points)
input bool   InpShowDailyOpen = true; // Draw a line at today's opening price

input group "Notifications"
input bool   InpAlertPopup   = true; // Show a terminal Alert() popup
input bool   InpAlertPush    = false; // Send a push notification to mobile
input bool   InpAlertSound   = true;  // Play a sound when a level is reached
input string InpAlertSoundFile = "alert.wav";

input group "Appearance"
input color   InpColorHigh    = clrDeepSkyBlue;
input color   InpColorLow     = clrOrangeRed;
input color   InpColorOpen    = clrGray;
input color   InpColorHitHigh  = clrLime;
input color   InpColorHitLow   = clrRed;
input int     InpFontSize     = 9;
input int     InpLineWidth    = 1;

//--- Internal state -----
double   g_adrHighLevel = 0.0;
double   g_adrLowLevel  = 0.0;
double   g_todayOpen    = 0.0;
datetime g_todayStart   = 0;
bool     g_highReached  = false;
bool     g_lowReached   = false;
double   g_pointFactor  = 1.0;    // converts points -> "pips" on 5/3-digit brokers
```

```

string  g_prefix;           // object-name prefix, keeps this indicator's objects unique

//+-----+
int OnInit()
{
    g_prefix = "ADR_" + _Symbol + "_";

    // 5-digit / 3-digit brokers quote one extra decimal - 10 points = 1 pip
    g_pointFactor = (_Digits == 3 || _Digits == 5) ? 10.0 : 1.0;

    CalculateADR();
    DrawLevels();

    return(INIT_SUCCEEDED);
}

//+-----+
void OnDeinit(const int reason)
{
    ObjectsDeleteAll(0, g_prefix);
    Comment("");
}

//+-----+
int OnCalculate(const int rates_total,
                const int prev_calculated,
                const datetime &time[],
                const double &open[],
                const double &high[],
                const double &low[],
                const double &close[],
                const long &tick_volume[],
                const long &volume[],
                const int &spread[])
{
    datetime barDayStart = iTime(_Symbol, PERIOD_D1, 0);

    // A new trading day has started - recalculate ADR and redraw fresh lines
    if(barDayStart != g_todayStart)
    {
        g_todayStart = barDayStart;
        g_highReached = false;
        g_lowReached = false;
        CalculateADR();
        DrawLevels();
    }

    UpdateLiveInfo();

    return(rates_total);
}

//+-----+
//| Average the previous N completed days' open-to-high and          |
//| open-to-low distances, then project them from today's open        |
//+-----+
void CalculateADR()
{
    double sumUp = 0.0, sumDown = 0.0;
    int counted = 0;

    // Shift 1 = yesterday (last completed day) ... InpADRDays = oldest day used
    for(int i = 1; i <= InpADRDays; i++)
    {
        double dOpen = iOpen(_Symbol, PERIOD_D1, i);
        double dHigh = iHigh(_Symbol, PERIOD_D1, i);
    }
}

```

```

double dLow = iLow(_Symbol, PERIOD_D1, i);

if(dOpen == 0.0)
    continue; // not enough history yet

sumUp += (dHigh - dOpen);
sumDown += (dOpen - dLow);
counted++;
}

if(counted == 0)
{
    Print("ADR_Levels: not enough daily history for ", InpADRDays, " days.");
    return;
}

double avgUp = sumUp / counted;
double avgDown = sumDown / counted;
double buffer = InpBufferPoints * _Point;

g_todayOpen = iOpen(_Symbol, PERIOD_D1, 0);
g_adrHighLevel = g_todayOpen + avgUp + buffer;
g_adrLowLevel = g_todayOpen - avgDown - buffer;
}

//+-----+
//| Draw the daily-open, ADR-high and ADR-low lines as dotted rays |
//| starting at market open and extending across the trading day   |
//+-----+
void DrawLevels()
{
    datetime startTime = g_todayStart;
    datetime endTime = startTime + PeriodSeconds(PERIOD_D1) * 2; // extend across the day

    DrawDottedLine(g_prefix + "High", startTime, endTime, g_adrHighLevel, InpColorHigh);
    DrawDottedLine(g_prefix + "Low", startTime, endTime, g_adrLowLevel, InpColorLow);

    if(InpShowDailyOpen)
        DrawDottedLine(g_prefix + "Open", startTime, endTime, g_todayOpen, InpColorOpen);

    UpdateLabel(g_prefix + "HighLabel", g_adrHighLevel, InpColorHigh,
        StringFormat("ADR HIGH (%d-day) %s", InpADRDays, DoubleToString(g_adrHighLevel, _Digits)));
    UpdateLabel(g_prefix + "LowLabel", g_adrLowLevel, InpColorLow,
        StringFormat("ADR LOW (%d-day) %s", InpADRDays, DoubleToString(g_adrLowLevel, _Digits)));
}

//+-----+
void DrawDottedLine(string name, datetime t1, datetime t2, double price, color clr)
{
    if(ObjectFind(0, name) < 0)
        ObjectCreate(0, name, OBJ_TREND, 0, t1, price, t2, price);
    else
        ObjectMove(0, name, 0, t1, price);

    ObjectSetInteger(0, name, OBJPROP_TIME, 1, t2);
    ObjectSetDouble(0, name, OBJPROP_PRICE, 1, price);
    ObjectSetInteger(0, name, OBJPROP_COLOR, clr);
    ObjectSetInteger(0, name, OBJPROP_STYLE, STYLE_DOT);
    ObjectSetInteger(0, name, OBJPROP_WIDTH, InpLineWidth);
    ObjectSetInteger(0, name, OBJPROP_RAY_RIGHT, false);
    ObjectSetInteger(0, name, OBJPROP_SELECTABLE, false);
    ObjectSetInteger(0, name, OBJPROP_HIDDEN, true);
}

//+-----+
void UpdateLabel(string name, double price, color clr, string text)

```

```

{
    if(ObjectFind(0, name) < 0)
        ObjectCreate(0, name, OBJ_TEXT, 0, TimeCurrent(), price);

    ObjectSetString(0, name, OBJPROP_TEXT, text);
    ObjectSetInteger(0, name, OBJPROP_TIME, TimeCurrent());
    ObjectSetDouble(0, name, OBJPROP_PRICE, price);
    ObjectSetInteger(0, name, OBJPROP_COLOR, clr);
    ObjectSetInteger(0, name, OBJPROP_FONTSIZE, InpFontSize);
    ObjectSetInteger(0, name, OBJPROP_ANCHOR, ANCHOR_LEFT_LOWER);
    ObjectSetInteger(0, name, OBJPROP_SELECTABLE, false);
    ObjectSetInteger(0, name, OBJPROP_HIDDEN, true);
}

//+-----+
//| Runs on every tick: checks for level breaks and refreshes the |
//| on-chart status panel with pips remaining to each level      |
//+-----+
void UpdateLiveInfo()
{
    double bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);

    double pipsToHigh = (g_adrHighLevel - bid) / _Point / g_pointFactor;
    double pipsToLow  = (bid - g_adrLowLevel) / _Point / g_pointFactor;

    //--- breakout detection -----
    if(!g_highReached && bid >= g_adrHighLevel)
    {
        g_highReached = true;
        NotifyLevelReached("ADR HIGH", g_adrHighLevel);
        UpdateLabel(g_prefix + "HighLabel", g_adrHighLevel, InpColorHitHigh,
            StringFormat("ADR HIGH +%s pts - REACHED", DoubleToString((g_adrHighLevel - g_todayOpen) / _Point, 1)));
    }

    if(!g_lowReached && bid <= g_adrLowLevel)
    {
        g_lowReached = true;
        NotifyLevelReached("ADR LOW", g_adrLowLevel);
        UpdateLabel(g_prefix + "LowLabel", g_adrLowLevel, InpColorHitLow,
            StringFormat("ADR LOW -%s pts - REACHED", DoubleToString((g_todayOpen - g_adrLowLevel) / _Point, 1)));
    }

    //--- on-chart status panel -----
    string panel = StringFormat(
        "ADR (%d-Day) Indicator\n" +
        "Daily Open : %s\n" +
        "ADR High   : %s (%s)\n" +
        "ADR Low    : %s (%s)\n" +
        "Pips to High: %s\n" +
        "Pips to Low : %s",
        InpADRDays,
        DoubleToString(g_todayOpen, _Digits),
        DoubleToString(g_adrHighLevel, _Digits), (g_highReached ? "REACHED" : "pending"),
        DoubleToString(g_adrLowLevel, _Digits), (g_lowReached ? "REACHED" : "pending"),
        (g_highReached ? "-" : DoubleToString(MathMax(pipsToHigh, 0), 1)),
        (g_lowReached ? "-" : DoubleToString(MathMax(pipsToLow, 0), 1))
    );

    Comment(panel);
}

//+-----+
void NotifyLevelReached(string levelName, double levelPrice)
{
    string msg = StringFormat("%s: %s level reached at %s (%s)",

```

```
        _Symbol, levelName,  
        DoubleToString(levelPrice, _Digits),  
        TimeToString(TimeCurrent(), TIME_DATE | TIME_MINUTES));  
  
    if(InpAlertPopup) Alert(msg);  
    if(InpAlertPush)  SendNotification(msg);  
    if(InpAlertSound) PlaySound(InpAlertSoundFile);  
  
    Print(msg);  
}  
//+-----+
```

Appendix A — ADR_Levels.mq5, the complete indicator source referenced in Part 8 and Part 9.